

Approches classiques de l'IA

Chapitre 4 : Recherche locale et métaheuristiques

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 4. Recherche locale et métaheuristiques

- 1 Algorithmes de recherche locale
- 2 Métaheuristiques
- 3 Métaheuristiques de type “solution unique”
- 4 Métaheuristiques de type “population”
- 5 Résumé du chapitre

Algorithmes de recherche locale

Algorithmes de recherche locale

Stratégies d'exploration vues au Chapitre 3 :

- une solution = une **séquence** d'actions (le **chemin** de l'état initial à un état objectif est important)
- exploration **systématique** de l'espace d'états (**graphe d'états**)
- en mémoire : un ou plusieurs chemins + ce qui a déjà été exploré sur ce(s) chemin(s) (peut avoir une **complexité en espace exponentielle**)

Algorithmes de recherche locale : si le **chemin** vers une solution n'a pas d'importance (seule la solution importe) :

- permet une exploration **locale** du graphe d'états
- utilisation de la mémoire très faible (souvent un **espace mémoire constant**)

Types de problèmes

Dans beaucoup de problèmes seul importe l'état objectif en lui-même, et pas le *chemin* pour y arriver.

En particulier : les **problèmes d'optimisation** (= quelle est la meilleure solution parmi toutes les solutions réalisables ?)

Quantité de problèmes de ce type (e.g., en mathématiques, en informatique, en recherche opérationnelle) et beaucoup d'entre eux ont un nombre exponentiel de solutions réalisables. Ils sont parmi les problèmes les plus difficiles à résoudre (NP-complets).

Types de problèmes

Types de problème où seule la solution importe :

- **problème d'optimisation** : trouver une solution **optimale** (qui minimise ou maximise une **fonction objectif**)
- **problème de satisfaction de contraintes** : trouver une solution qui **respecte** certaines contraintes

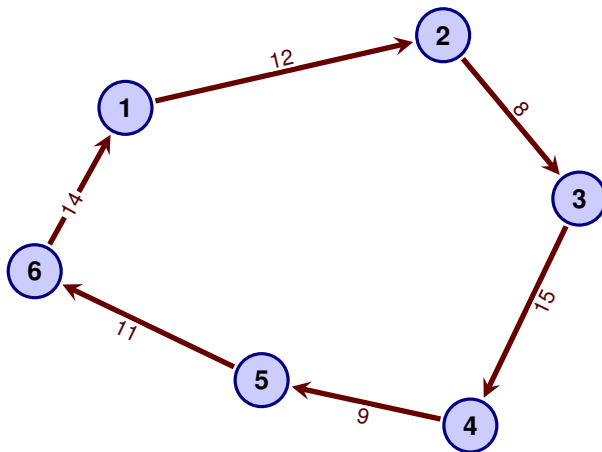
Remarques :

- un problème de minimisation peut être ramené à un problème de maximisation (ou inversement) en multipliant la fonction objectif par -1
- un problème de satisfaction de contraintes peut être ramené à un problème de minimisation où la fonction objectif compte (ou utilise) le nombre de contraintes violées

Exemple : Travelling Salesperson Problem (TSP)

- **Instance (données) :**
 - ▶ Un ensemble de n villes
 - ▶ Une matrice de coûts (ou distances) entre chaque paire de villes
- **Solution réalisable :** un tour complet visitant chaque ville une et une seule fois (et revient au point de départ)
- **Objectif : Minimiser** la somme des distances du tour
- **Taille de l'espace d'états :** nombre de tours possibles : $\frac{(n-1)!}{2}$
(ex : avec 20 villes $\approx 6 \times 10^{16}$ possibilités).

Exemple : Travelling Salesperson Problem (TSP)

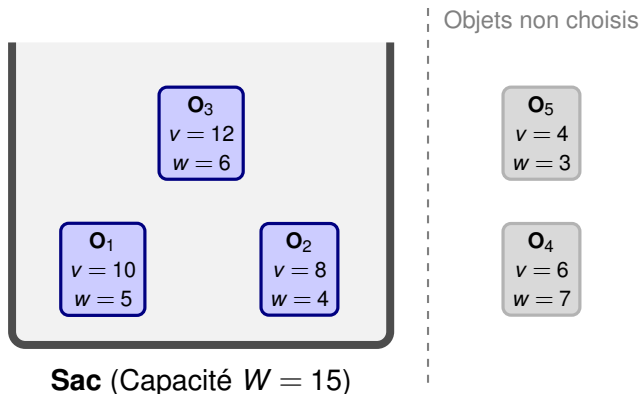


Exemple d'une solution réalisable d'un TSP avec 6 villes
(Coût total = $12 + 8 + 15 + 9 + 11 + 14 = 69$)

Exemple : Problème du sac à dos (Knapsack - KP)

- Instance (données) :
 - ▶ Un ensemble de n objets
 - ▶ Pour chaque objet i : une valeur v_i et un poids w_i
 - ▶ Une capacité maximale W du sac
- Solution réalisable : un sous-ensemble d'objets dont la somme des poids ne dépasse pas W
- Objectif : **Maximiser** la somme des valeurs des objets sélectionnés
- Taille de l'espace d'états : nombre de sous-ensembles possibles : 2^n
(ex : avec 60 objets $\approx 1,15 \times 10^{18}$ possibilités).

Exemple : Problème du sac à dos (Knapsack - KP)

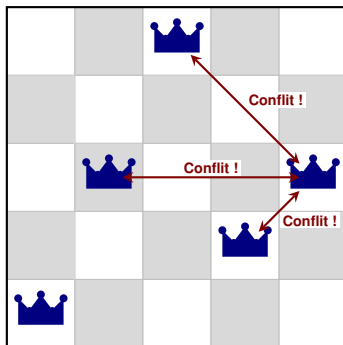


Exemple de solution réalisable ($W_{total} = 5 + 4 + 6 = 15 \leq 15$)
 (Valeur totale = $10 + 8 + 12 = \mathbf{30}$)

Exemple : Problème des n -reines (n -Queens)

- Instance (données) :
 - ▶ Un entier n (taille de l'échiquier $n \times n$ et nombre de reines)
- Solution candidate : Un placement de n reines sur l'échiquier, avec une reine par colonne
- Objectif : **Minimiser** le nombre de conflits (paires de reines s'attaquant sur une ligne ou une diagonale)
- Taille de l'espace d'états : si une seule reine par colonne : n^n états
(ex : pour un échiquier standard $n = 8$; $8^8 \approx 16$ millions d'états).

Exemple : Problème des n -reines (n -Queens)



Exemple de solution candidate pour $n = 5$

Coût = 3 (3 paires de reines en conflit)

Algorithmes de recherche locale

Idée : **amélioration itérative** d'une solution "courante" (unique) par exploration du **voisinage** de celle-ci.

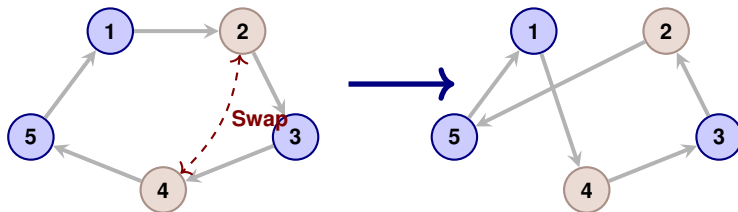
- L'opérateur de **voisinage** définit comment la solution courante peut être (légèrement) modifiée pour trouver un ensemble de solutions voisines.
- Comme au chapitre 3, on peut voir ces problèmes comme étant un parcours d'un **graphe d'états**
 - ▶ les nœuds correspondent aux états
 - ▶ les arcs correspondent à l'application de l'opérateur de voisinage
- Contrairement au chapitre 3, il n'est pas nécessaire de retenir le chemin depuis la solution initiale (**espace mémoire constant**), d'où le nom de recherche locale

Exemple d'opérateurs de voisinage pour le TSP

Opérateur swap : échanger la position de deux villes i et j dans la séquence

Taille du voisinage : $\frac{n(n-1)}{2} \in O(n^2)$ voisins possibles

Exemple : un voisin possible via “swap 2 et 4”



Avant : 1 - 2 - 3 - 4 - 5

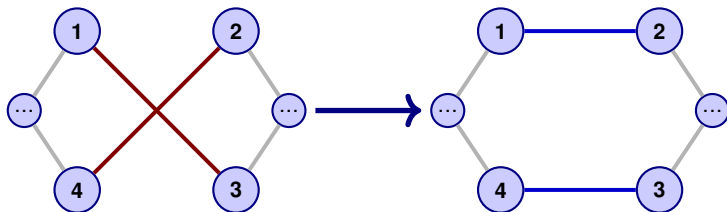
Après : 1 - 4 - 3 - 2 - 5

Exemple d'opérateurs de voisinage pour le TSP

Opérateur 2-opt : supprimer deux arêtes non adjacentes et reconnecter

Taille du voisinage : $O(n^2)$ voisins possibles (n arêtes)

Exemple : suppression des croisements



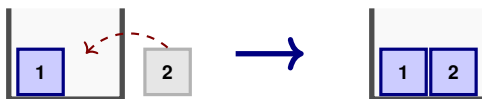
Avant : 1 - 3 - ... - 2 - 4 - ... - 1

Après : 1 - 2 - ... - 3 - 4 - ... - 1

Exemple d'opérateurs de voisinage pour le Sac à Dos

Opérateur Flip : déplacer un objet de l'extérieur du sac vers l'intérieur (ou inversement)

Taille du voisinage : n voisins



Opérateur Swap : échanger un objet du sac avec un objet extérieur

Taille du voisinage : $O(n^2)$ voisins.

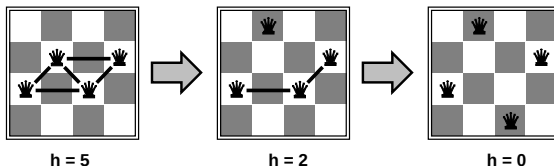


Exemple d'opérateur de voisinage pour les n -Reines

Opérateur : déplacer une reine sur une autre ligne de sa propre colonne.

Taille du voisinage : $n(n - 1)$ voisins (chaque reine a $n - 1$ autres positions possibles).

Exemple : deux voisins successifs



Hill-climbing

Algorithme de recherche locale le plus basique : le Hill-climbing

```

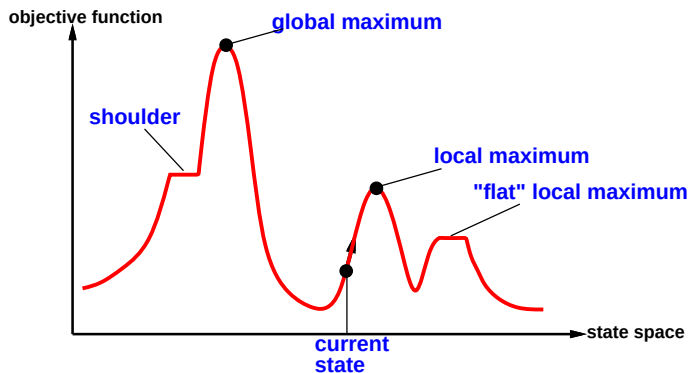
function HILL-CLIMBING(problem) returns a state that is a local maximum
  current ← MAKE-NODE(problem.INITIAL-STATE)
  loop do
    neighbor ← a highest-valued successor of current
    if neighbor.VALUE ≤ current.VALUE then return current.STATE
    current ← neighbor
  end

```

Intuition : Ascension d'une montagne, dans le brouillard complet : on ne voit qu'à 1m ; on choisit le pas qui nous élève le plus ; et on s'arrête quand aucun nouveau pas ne peut plus nous faire monter.

Hill-climbing

Intuition de “paysage”



Hill climbing

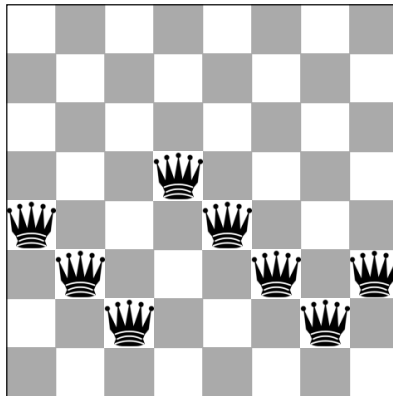
- Egalement appelé recherche locale **gloutonne** (greedy local search).
- Fonctionne souvent assez bien en pratique.
- **Mais** pas complet ni optimal : retourne le premier maximum local trouvé

Hill climbing : exemple (8-reines)

Résoudre le problème des 8-reines par Hill climbing :

- Une solution = placer les 8 reines sur l'échiquier en en plaçant seulement 1 par colonne
- Opération de voisinage = déplacer une reine sur un autre carré de sa propre colonne
- Chaque état à donc $8 \times 7 = 56$ successeurs (voisins)
- Fonction de coût h = nombre de paires de reines en conflit
- Si $h = 0$, alors problème résolu

Hill climbing : exemple (8-reines)



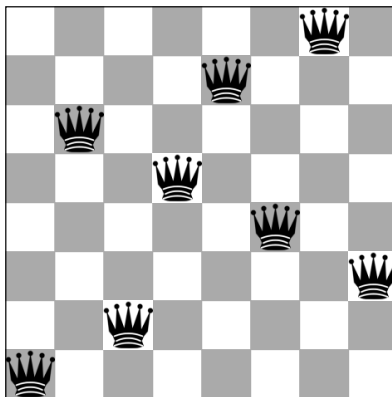
Une solution initiale avec 17 conflits ($h = 17$).

Hill climbing : exemple (8-reines)

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14		13	16	13	16
	14	17	15		14	16	16
17		16	18	15			
18	14		15	15	14		16
14	14	13	17	12	14	12	18

Calcul de la valeur de h pour chacun des 56 successeurs (= que devient h si on bouge une reine sur sa propre colonne ?). Le minimum est 12.

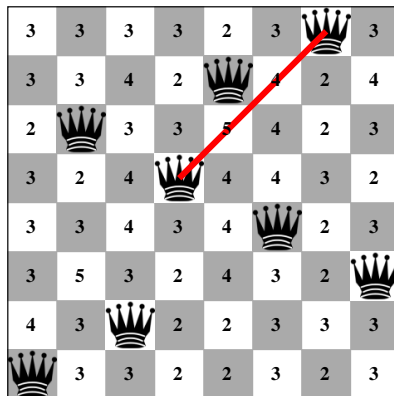
Hill climbing : exemple (8-reines)



Après 5 mouvements^a, l'algorithme trouve ce minimum local ($h = 1$, il reste donc un conflit, et tous les successeurs ont une valeur de $h \geq 2$).

a. Voir démonstration avec code en Python.

Hill climbing : exemple (8-reines)



Après 5 mouvements^a, l'algorithme trouve ce minimum local ($h = 1$, il reste donc un conflit, et tous les successeurs ont une valeur de $h \geq 2$).

a. Voir démonstration avec code en Python.

Métaheuristiques

Métaheuristiques

Comment gérer le problème des optima locaux qui peut apparaître dans un algorithme de recherche locale ?

Il existe des méthodes diverses, parfois inspirées de la nature. On les appelle des **métaheuristiques**.

Attention à la **terminologie** concernant les méthodes (algorithmes) : algorithme exact, heuristique, métaheuristique, algorithme d'approximation. . . que veulent dire précisément ces mots ?

Un peu de vocabulaire

On se place dans le contexte présenté au début de ce chapitre : problèmes dont seule la solution importe. Principalement, problèmes d'optimisation.

En fonction de la complexité du problème, il peut être résolu par

- une méthode **exacte** : trouve une solution optimale et garantit son optimalité ;
- une méthode **approchée** : trouve une solution supposée proche de la solution optimale — avec un temps de calcul raisonnable — mais sans garantie sur l'optimalité.

Un peu de vocabulaire

Quelques exemples de méthodes **exactes** célèbres :

- (en IA) : algorithmes d'exploration A^* ;
- (en RO) : programmation dynamique, branch and bound, branch and cut.

⇒ ne peuvent être appliquées que sur des instances très petites pour des problèmes difficiles (NP-difficiles)

Un peu de vocabulaire

Parmi les méthodes approchées on distingue 2 sous-classes :

- **heuristique** = méthode qui trouve une “bonne” solution rapidement, même sur des (très) grandes instances de problèmes difficiles, mais sans garantie sur l’optimalité.
- **algorithme d’approximation** = heuristique + garantie théorique sur l’approximation (**facteur d’approximation**, par ex. “dans le pire des cas, trouve une solution 50% moins bonne que la solution optimale”).

Remarques :

- **Attention, ne pas confondre** ! La “fonction d’heuristique” $h(n)$ qui est utilisée dans A^* (voir chap. 3) = fonction apportant de l’information spécifique au problème. Mais A^* n’est pas une heuristique : si h est consistante, c’est une méthode exacte.
- les algorithmes d’approximation sont vu dans le cours “Graphes et IA” (option Master).

Un peu de vocabulaire

Parmi les heuristiques, on distingue 2 types :

- **heuristique spécifique** = conçu et adapté pour résoudre un problème spécifique et/ou une instance spécifique.
- **métaheuristique** = algorithmes généralisables qui peuvent être appliqués pour résoudre presque tout problème d'optimisation.

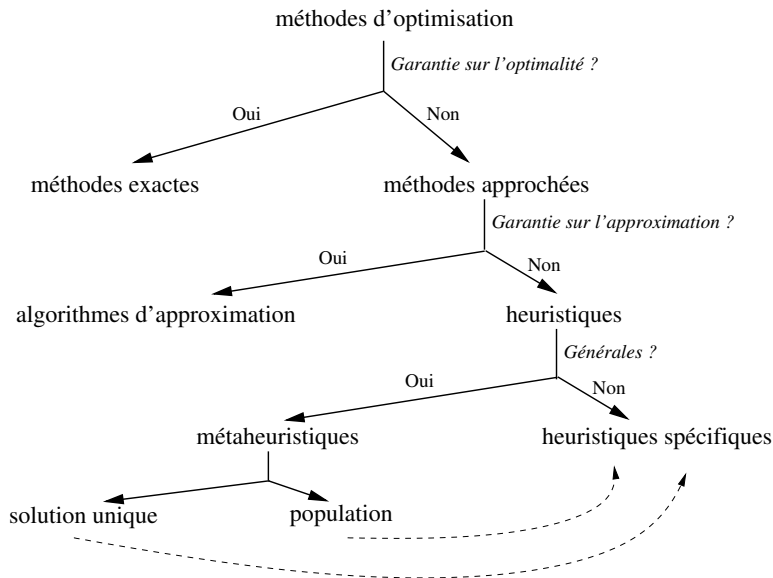
On peut donc voir les méthaheuristiques comme des méthodes générales qui peuvent être utilisées comme guide stratégique pour concevoir des heuristiques permettant de résoudre une problème d'optimisation spécifique.

Un peu de vocabulaire

Enfin, parmi les métaheuristiques, on distingue encore 2 types :

- **solution unique** (single-based metaheuristics) = manipule et transforme une seule solution (par ex., algorithmes de recherche locale, hill climbing, simulated annealing, recherche Tabou)
- **population** (population-based metaheuristics) = manipule et fait évoluer une population complète de solutions (par ex., algorithmes génétiques, colonies de fourmis).

Un peu de vocabulaire



Principaux concepts communs aux métaheuristiques

Il y a quantité de métaheuristiques, mais elles doivent toutes répondre aux mêmes questions :

- **représentation** : comment représenter / encoder une solution ?
- comment définir la **fonction objectif** ?
- comment gérer les **contraintes** ?
- comment **régler les paramètres** ?
- comment **analyser les performances** ?

En ce qui concerne les 2 derniers points : voir livre “Metaheuristics : from design to implementation” de Talbi.

Concepts communs aux métaheuristiques : représentation

Comment encoder une solution ?

Concepts communs aux métaheuristiques : représentation

Comment encoder une solution ?

Quelques représentations classiques :

- **codage binaire** (par ex. problème du sac à dos)

1 0 0 0 1 0 1 1 0 1

- **vecteur de valeurs discrètes** (par ex. problèmes d'assignation et de localisation)

5 7 6 6 2 1 4

- **permutation** (par ex. TSP, problèmes d'ordonnancement)

5 2 7 1 3 4 6

- **vecteur de valeurs réelles** (par ex. optimisation continue)

1.14 5.65 9.45 3.78

Concepts communs aux métaheuristiques : représentation

Le choix de la représentation joue un rôle majeur en terme d'efficacité car va influencer la **taille de l'espace d'états** : étape essentielle.

Exemple : problème des 8-reines

- vecteur de 8 coordonnées dans le plan $x = (p_1, p_2, \dots, p_8)$ où $p_i = (x_i, y_i)$ représente la position de la reine i dans le repère cartésien. Le nombre de solutions possibles (taille de l'espace d'états) $= 64^8 \simeq 2.8 \times 10^{14}$.
- si on restreint le nombre de reines par colonne (ou ligne) à 1 $= 8^8 \simeq 1.6 \times 10^7 \Rightarrow$ c'est ce qu'on a fait précédemment, via un vecteur de valeurs discrètes.

Concepts communs aux métaheuristiques : représentation

Exemple : problème des 8-reines

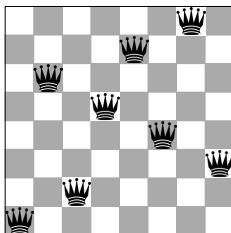
Y a-t-il moyen de faire mieux ? Par exemple en restreignant le nombre de reines par colonne **et** par ligne à 1 ?

Concepts communs aux métaheuristiques : représentation

Exemple : problème des 8-reines

Y a-t-il moyen de faire mieux ? Par exemple en restreignant le nombre de reines par colonne **et** par ligne à 1 ?

Oui, en représentant une solution par une permutation, ce qui réduit la taille de l'espace à $n!$ ($8! = 40320$).



Représentation :

$$\pi = \boxed{7 \ 5 \ 2 \ 4 \ 6 \ 8 \ 3 \ 1}$$

où $\pi[i] = j$ signifie “la reine de la ligne i se trouve en colonne j ”.

Concepts communs aux métaheuristiques : fonction objectif

Comment définir la fonction objectif ou la fonction de coût ?

Concepts communs aux métaheuristiques : fonction objectif

Comment définir la fonction objectif ou la fonction de coût ? Souvent évident pour beaucoup de problèmes d'optimisation (par ex. somme des distances).

Cependant on peut définir plusieurs types de fonctions objectifs pour un même problème. Cela a une influence sur le comportement des métaheuristiques car les fonctions objectifs “dessinent” les hauteurs du paysage.

Concepts communs aux métaheuristiques : fonction objectif

Comment définir la fonction objectif ou la fonction de coût ? Souvent évident pour beaucoup de problèmes d'optimisation (par ex. somme des distances).

Cependant on peut définir plusieurs types de fonctions objectifs pour un même problème. Cela a une influence sur le comportement des métaheuristiques car les fonctions objectifs “dessinent” les hauteurs du paysage.

En général, on essaye d'avoir une fonction objectif qui réalise une bonne **différenciation** des solutions $\Rightarrow$ moins de plateaux.

Exemple (problème des n -reines) : “nombre de paires de reines en conflit” (= entier de 0 à $\frac{n^2-n}{2}$) meilleur que “nombre de reines qui font partie d'un conflit” (= entier de 0 à n).

Concepts communs aux métaheuristiques : contraintes

Comment gérer les contraintes ?

La plupart des problèmes d'optimisation possèdent des **contraintes** et ce n'est pas toujours trivial de gérer ces contraintes. Les stratégies les plus classiques :

- **Rejet** : seules les solutions réalisables sont conservées pendant la recherche. **Problème** : peut produire un graphe d'états **non-connexe**
- **Pénalisation** : ajouter une pénalité dans la fonction objectif pour les contraintes violées.

Exercice en séance : créer une fonction objectif pour le sac à dos qui

- pénalise les solutions non réalisables (pour éviter de les rejeter complètement)
- s'assure que toute solution réalisable est meilleure qu'une solution non réalisable

Métaheuristiques de type “solution unique”

S-métaheuristiques

La recherche locale gloutonne (Hill-climbing) est un exemple de métaheuristique utilisant une solution courante unique.

On parle de **S-métaheuristique** (single-based metaheuristic).

Exemples de S-métaheuristiques : recuit simulé, recherche "Tabou", recherche locale itérative, recherche par voisinage variable, recherche locale guidée, GRASP, etc.

S-métaheuristiques

D'une manière informelle :

```

function S-METAHEURISTIC(problem) returns a solution state
  current ← MAKE-NODE(problem.INITIAL-STATE)
  t ← 0
  repeat until stopping criteria satisfied
    N ← GENERATE-NEIGHBORHOOD(current)
    current ← SELECT(N)
    t ← t + 1
  return current

```

Concepts communs aux S-métaheuristiques

Qu'est-ce qui distingue les différentes S-métaheuristiques ?

- **diversification** : stratégie utilisée pour **sortir d'optima locaux** dans l'espoir de trouver un optimum global (ou un meilleur optimum local).
- **intensification** : stratégie utilisée pour **concentrer** la recherche locale autour de zones "prometteuses"

Quelles sont les questions communes aux S-métaheuristiques ?

- comment définir le **voisinage** ?
- quelle **solution initiale** choisir ?
- comment **évaluer de façon incrémentale** un voisinage ?

Voisinage

Comment définir le voisinage d'une solution ?

Définition

Soit S l'ensemble des solutions. Une fonction de **voisinage** N est une fonction qui assigne à chaque solution $s \in S$ un ensemble de solutions $N(s) \subset S$.

Dans ce cours, nous nous intéressons à des problèmes d'optimisation **discrets**.

Dans un problème d'optimisation discret, le voisinage $N(s)$ d'une solution l'ensemble des solutions obtenue en appliquant une **opération de voisinage** qui modifie (légèrement) s .

⇒ cela dépend de la manière dont sont **représentées** les solutions

Exemples de voisinage : codage binaire

Représentation en codage binaire : **flip** (on échange une seule valeur, i.e. un 1 devient un 0 ou l'inverse)

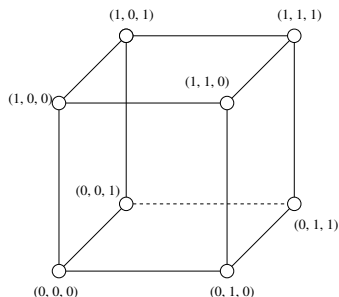
Pour un vecteur binaire de taille n , la taille du voisinage est alors égale à n .

Exemples de voisinage : codage binaire

Représentation en codage binaire : **flip** (on échange une seule valeur, i.e. un 1 devient un 0 ou l'inverse)

Pour un vecteur binaire de taille n , la taille du voisinage est alors égale à n .

Exemple : sac à dos avec 3 objets



Exemples de voisinage : vecteur discret

Représentation via un vecteur de valeurs discrètes : modifie une seule valeur (extension du **flip**)

La taille d'un voisinage est alors égale à $(k - 1) \cdot n$ où k est la cardinalité de l'ensemble des valeurs possibles et n la taille du vecteur discret.

Exemple : problème des n -reines où x_i représente la "hauteur" de la reine i dans sa colonne : $k = 8$, $n = 8$ et le nombre de voisins est $7 \times 8 = 56$.

Exemples de voisinage : permutation

Représentation utilisant une permutation des nombres de 1 à n : un opérateur de voisinage classique est défini par l'échange (**swap**) de 2 éléments de la permutation.

La taille d'un voisinage est alors égale à $\frac{n(n-1)}{2}$.

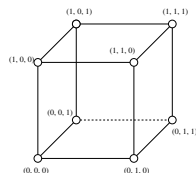
Exemple : problème des n -reines utilisant une permutation comme représentation. Chaque solution possède $\frac{56}{2} = 28$ voisins (il y a 28 paires de reines candidates pour un swap).

Voisinages en fonction du problème

La qualité d'un voisinage dépend aussi du problème (pas seulement de la représentation). Un voisinage devrait :

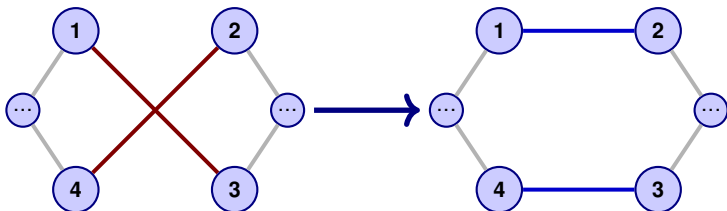
- permettre d'atteindre n'importe quelle autre solution en un nombre fini d'étapes (**connexité**)
- engendrer un petit nombre (polynomial) de voisins avec des valeurs proches pour la fonction objectif (**localité**)

Exemple (connexité) : pour un vecteur binaire (par ex. problème du sac à dos), le flip est adapté pour atteindre toutes les solutions possibles, contrairement au swap.



Voisinages en fonction du problème

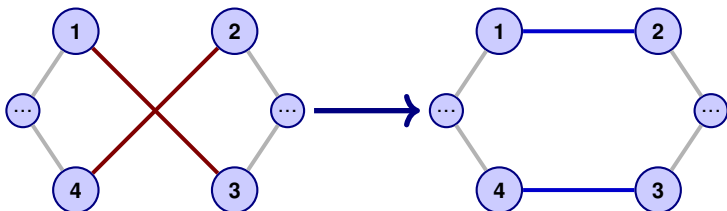
Exemple (localité) : représentation par permutation utilisée pour le TSP et voisinage = "2-opt".



Variations faibles car nombre de voisins polynomial ($O(n^2)$) et seules 2 distances sont modifiées dans le calcul de la somme des distances (localité forte)

Voisinages en fonction du problème

Remarque : le "2-opt" peut avoir des implications fortes sur l'**ordre relatif** des villes dans le tour, mais ce n'est pas important pour le TSP si les distances sont symétriques.



Par contre, imaginons qu'une permutation soit utilisée pour représenter un **problème d'ordonnement**. Le 2-opt a alors une **localité faible** car l'**ordre relatif** dans la séquence y est très important. Dans ce cas, le swap est sans doute mieux adapté car il modifie l'ordre relatif plus légèrement.

Solution initiale

Quelle solution initiale choisir ?

Deux stratégies principales : aléatoirement ou en utilisant une heuristique (gloutonne).

Ce choix influence la qualité de la solution initiale et le temps de calcul pour la générer :

- une solution aléatoire est rapide à générer, mais la métaheuristique peut utiliser un grand nombre d'itérations pour converger ;
- une solution initiale gloutonne est plus lente à générer mais permet en général d'accélérer la recherche (mais ce n'est pas toujours le cas).

Evaluation incrémentale

Comme évaluer de façon incrémentale un voisinage ?

La plupart du temps, l'évaluation de la fonction objectif est la partie la plus coûteuse en temps de calcul (car souvent répétée).

Méthode naïve : évaluer la fonction objectif pour tous les voisins dans $N(s)$.

Evaluation incrémentale

Comme évaluer de façon incrémentale un voisinage ?

La plupart du temps, l'évaluation de la fonction objectif est la partie la plus coûteuse en temps de calcul (car souvent répétée).

Méthode naïve : évaluer la fonction objectif pour tous les voisins dans $N(s)$.

Evaluation incrémentale : peut-on évaluer $\Delta(s, o)$ pour une solution s et une opération de voisinage o (transformant s en s') tel que

$$f(s') = f(s) + \Delta(s, o) ?$$

$\Rightarrow$ utile si la complexité du calcul de $\Delta(s, o)$ est meilleure que celle du calcul direct de $f(s')$.

Evaluation incrémentale

Exemple : TSP représenté par une permutation π et voisinage "2-opt".
L'accès à la distance entre 2 villes données est en temps constant (via une matrice des distances D).

Coût pour évaluer un voisinage complet de taille $O(n^2)$:

$$O(n^3) \implies O(n^2)$$

Evaluation incrémentale

Exemple : TSP représenté par une permutation π et voisinage "2-opt".
L'accès à la distance entre 2 villes données est en temps constant (via une matrice des distances D).

- évaluation **naïve** : parcours linéaire = $O(n)$

$$f = \sum_{i=1}^{n-1} D[\pi_i][\pi_{i+1}] + D[\pi_n][\pi_1].$$

- évaluation **incrémentale** : seules 4 distances changent = $O(1)$

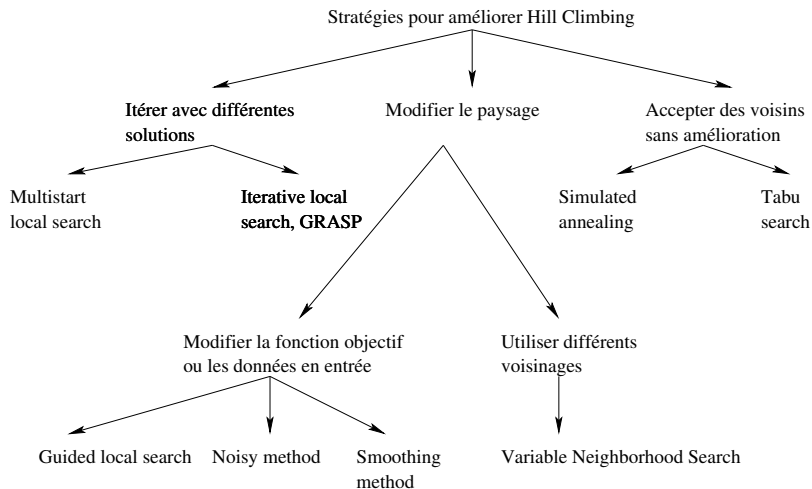
$$\Delta f = D[\pi_i][\pi_j] + D[\pi_{i+1}][\pi_{j+1}] - D[\pi_i][\pi_{i+1}] - D[\pi_j][\pi_{j+1}]$$

$$f \leftarrow f + \Delta f$$

Coût pour évaluer un voisinage complet de taille $O(n^2)$:

$$O(n^3) \implies O(n^2)$$

S'échapper des optima locaux



Petit catalogue de S-métaheuristiques

Parmi toutes ces S-métaheuristiques, on présente rapidement ci-après :

- Simulated Annealing
- Tabu Search
- Iterated Local Search
- Variable Neighborhood Search

Simulated annealing (SA — recuit simulé)

[Kirkpatrick et al., 1983 ; Cerny, 1985]

Idée :

- sortir d'optima locaux en autorisant certains "mauvais mouvements" ;
- **mais** le faire graduellement : diminuer progressivement la taille et la fréquence de ces mouvements.

Inspiration (métallurgie) : processus en cycles où l'on chauffe un matériau (métal, verre) à très haute température, puis on le refroidit lentement, pour atteindre une cristallisation solide du matériau.

Simulated annealing (SA — recuit simulé)

[Kirkpatrick et al., 1983 ; Cerny, 1985]

Idée :

- sortir d'optima locaux en autorisant certains "mauvais mouvements" ;
- **mais** le faire graduellement : diminuer progressivement la taille et la fréquence de ces mouvements.

Inspiration (métallurgie) : processus en cycles où l'on chauffe un matériau (métal, verre) à très haute température, puis on le refroidit lentement, pour atteindre une cristallisation solide du matériau.

Intuition : balle roulant sur le paysage des états (problème de minimisation). On "secoue" le paysage suffisamment fort pour que la balle sorte d'un minimum local mais pas trop pour qu'elle reste dans un minimum global $\Rightarrow$ On commence par secouer fort (température élevée), puis on secoue graduellement de moins en moins fort (température diminue).

Simulated annealing (SA — recuit simulé)

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

inputs: *problem*, a problem

schedule, a mapping from time to "temperature"

current ← MAKE-NODE(*problem*.INITIAL-STATE)

for $t \leftarrow 1$ **to** ∞ **do**

$T \leftarrow \text{schedule}(t)$

if $T = 0$ **then return** *current*

next ← a randomly selected successor of *current*

$\Delta E \leftarrow \text{next.VALUE} - \text{current.VALUE}$

if $\Delta E > 0$ **then** *current* ← *next*

else *current* ← *next* only with probability $e^{\Delta E/T}$

Note : version "maximisation".

Propriétés du Simulated Annealing (SA)

- SA choisit donc un voisin **au hasard**. Si celui-ci améliore la solution (ce qui arrive si $\Delta E > 0$), il est sélectionné d'office. Sinon, il est sélectionné avec une probabilité

$$p(\Delta E, T) = e^{\frac{\Delta E}{T}}.$$

(comme dans ce cas $\Delta E < 0$, on a bien un nombre compris entre 0 et 1 : c'est la distribution de Boltzmann)

- pour ΔE fixé, cette probabilité est croissante en T (proche de 0 quand T est petit et proche de 1 quand T est grand).
- pour T fixé, cette probabilité est décroissante en ΔE (si l'écart entre les 2 solutions est très mauvais, moins grande probabilité de choisir la solution).

Propriétés du Simulated Annealing (SA)

Il y a une garantie théorique de la convergence de SA vers la solution optimale si on utilise le schéma de refroidissement suivant :

$$T_k = \frac{\Gamma}{\log k},$$

où k est le nombre d'itérations et Γ est une constante (mais cela ne veut pas dire grand chose en pratique car la garantie est obtenue après un nombre infini d'itérations).

Variantes du Simulated Annealing (SA)

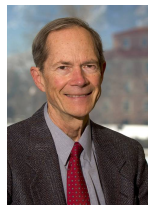
La méthode "recuit simulé" est largement utilisée (VLSI layout, airline scheduling, etc.). Différentes variantes existent, définies par :

- fonction calculant la probabilité d'accepter un mauvais mouvement ;
- le schéma de refroidissement.

D'autres métaheuristiques inspirées du SA : threshold accepting, great deluge algorithm, record-to-record travel, demon algorithms (but : augmenter la vitesse de la recherche du SA mais sans sacrifier la qualité des solutions).

Tabu Search (TS – Recherche Tabou)

[Glover, 1989] → une des S-métaheuristiques les plus répandues encore aujourd'hui



Idée : utilisation de la **mémoire** pour stocker une certaine information lors de la recherche

- comme dans une recherche locale classique, remplace la solution courante par un meilleur voisin ;
- **mais** accepte la "moins mauvaise" solution quand on se trouve dans un optimum local (tous les voisins sont moins bons) ;
- pour éviter les cycles, TS rejette les voisinages qui ont déjà été visités récemment (**tabu list**, en général listes de taille constante pour éviter l'explosion en mémoire).

Tabu Search (TS – Recherche Tabou)

La gestion de la **liste tabou** est cruciale :

- si trop petite : des cycles peuvent apparaître ;
- si trop grande : les complexités en temps et en espace peuvent devenir trop importantes.

La liste tabou est en général considérée comme la **mémoire à court terme**. D'autres mécanismes sophistiqués sont souvent ajoutés à une recherche tabou :

- **Intensification** (mémoire à **moyen** terme) : stocke les meilleures solutions (élite) pour leur donner une priorité lors de la recherche ;
- **Diversification** (mémoire à **long** terme) : stocke de l'information relative aux solutions visitées dans le but d'explorer des zones non visitées de l'espace d'états.

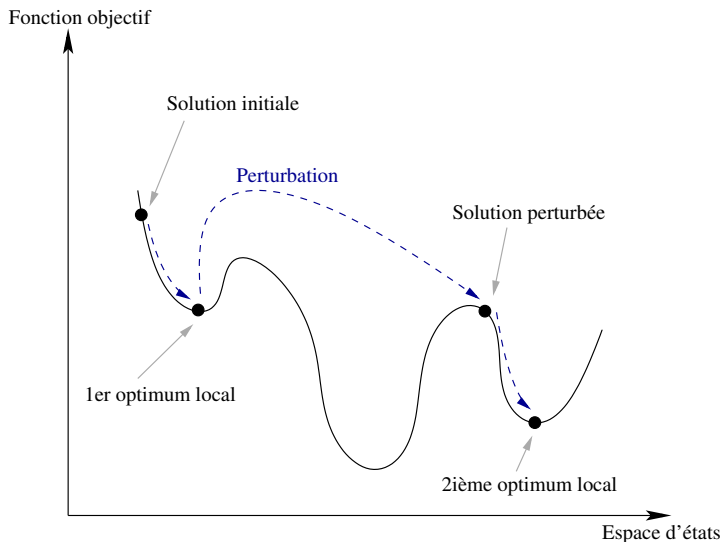
Iterated Local Search (ILS – Recherche locale itérée)

[Martin *et al.*, 1991]

Idée : améliorer la qualité d'optima locaux successifs

- la qualité d'un optimum local dépend de la solution initiale ;
- **multistart local search** classique : choisir aléatoirement plusieurs solutions initiales, réaliser une recherche locale pour chacune d'elles, et retourner le meilleur optimum local ;
- ILS : similaire à multistart **mais** chaque optimum local trouvé est légèrement "perturbé", puis réutilisé comme nouvelle solution initiale ;
- **perturbation** : plusieurs variantes mais peut être un voisin plus "éloigné" que les voisins utilisés lors de la recherche.

Iterated Local Search (ILS – Recherche locale itérée)



Variable Neighborhood Search (VNS – Recherche à voisinage variable)

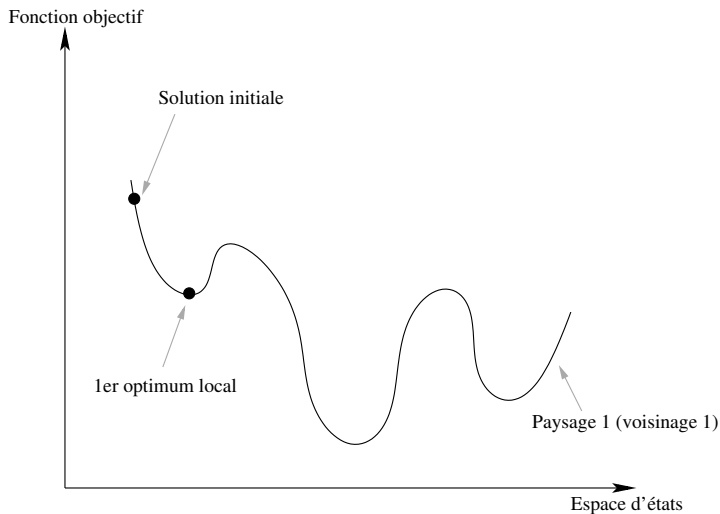
[Hansen et Mladenović,
1997]



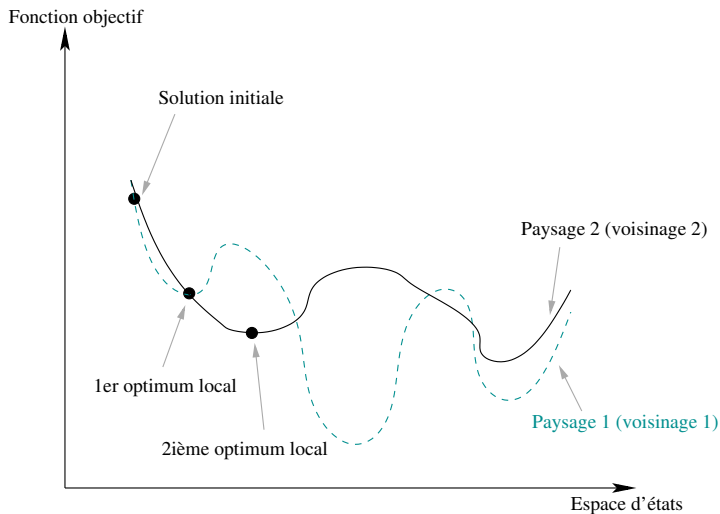
Idée : modifier le paysage

- explorer successivement un ensemble prédéfini de voisinages ;
- différents voisinages génèrent différents paysages ;
- un optimum local (qui n'est pas global) peut ne pas être un optimum local dans un paysage modifié ;
- possède des mécanismes d'intensification et de diversification.

Variable Neighborhood Search (VNS – Recherche à voisinage variable)



Variable Neighborhood Search (VNS – Recherche à voisinage variable)



Exemple d'application du VNS : AutoGraphiX

[Caporossi et Hansen,
2000]

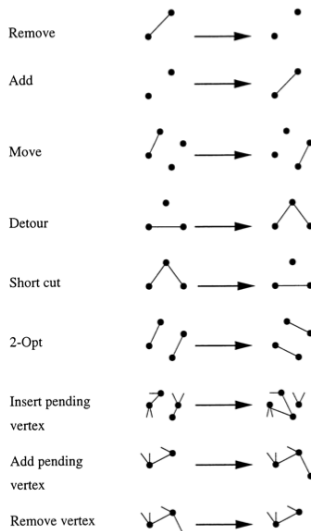


AGX est un logiciel d'aide à la découverte en théorie des graphes.

AGX utilise le VNS pour trouver des graphes (supposés) extrêmes pour un problème donné (par exemple : "Quels sont les graphes qui maximisent la somme des distances ?").

⇒ de nombreux résultats scientifiques ont été "suggérés" par AGX

Voisinages utilisés dans AutoGraphiX



Métaheuristiques de type "population"

P-métaheuristiques

Une métaheuristique qui utilise une **population** de solutions plutôt qu'une seule solution courante est une **P-métaheuristique** (population-based metaheuristic).

Chaque solution de la population est appelée un **individu**. L'analogie avec des processus naturels sur des individus faisant partie d'une population spécifique est fréquente (e.g., vocabulaire et algorithmes inspirés par la nature).

P-métaheuristiques

Une métaheuristique qui utilise une **population** de solutions plutôt qu'une seule solution courante est une **P-métaheuristique** (population-based metaheuristic).

Chaque solution de la population est appelée un **individu**. L'analogie avec des processus naturels sur des individus faisant partie d'une population spécifique est fréquente (e.g., vocabulaire et algorithmes inspirés par la nature).

Exemples de P-métaheuristiques : algorithmes évolutifs, scatter search, colonies de fourmis, colonies d'abeilles, systèmes immunitaires artificiels, etc.

Dans ce cours : présentation rapide de la local beam search et des algorithmes génétiques.

P-métaheuristiques

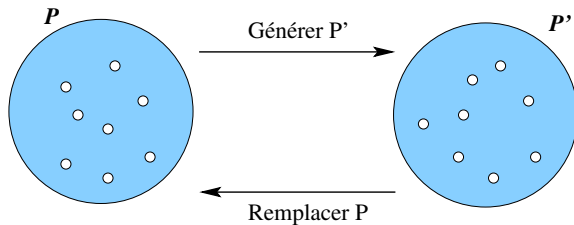
D'une manière informelle :

```

function P-METAHEURISTIC(problem) returns best solution(s) found
   $P \leftarrow \text{GENERATE-INITIAL-POPULATION}()$ 
   $t \leftarrow 0$ 
  repeat until stopping criteria satisfied
     $P' \leftarrow \text{GENERATE-NEW-POPULATION}(P)$ 
     $P \leftarrow \text{SELECT-POPULATION}(P \cup P')$ 
     $t \leftarrow t + 1$ 
  return SELECT-BEST-SOLUTIONS( $P$ )
  
```

Note : si problème d'optimisation, SELECT-BEST-SOLUTIONS ne retourne en général qu'une seule solution, la meilleure parmi la population finale.

P-métaheuristiques



Les P-metaheuristiques font évoluer une population P de solutions.

La taille de la population de P est constante.

Exemple : recherche locale en faisceau (local beam search)

- **Idée** : adaptation d'une exploration en gardant k états en mémoire plutôt qu'un seul :
 - ▶ démarre avec k états aléatoires ;
 - ▶ à chaque étape, **tous les successeurs des k états** sont générés ;
 - ▶ si l'un d'entre eux est un état objectif, l'algorithme s'arrête ;
 - ▶ sinon, on choisit les k meilleurs successeurs parmi **tous** les successeurs et on itère le processus.
- Cette approche
 - ≠ k explorations exécutées en parallèle de manière indépendante
 - ≈ k explorations parallèles **partageant de l'information** :
les explorations qui trouvent les meilleurs successeurs concentrent les explorations futures dans leur région de l'espace d'états.

Exemple : recherche locale en faisceau (local beam search)

- **Problème** : souvent, les k états finaux se retrouvent dans la même colline locale du paysage d'états
- **Variante (stochastic beam search)** :
choisir k successeurs aléatoirement, avec une préférence (probabiliste) à l'avantage des meilleurs
- Analogie avec la sélection naturelle

Concepts communs aux P-métaheuristiques

Qu'est-ce qui distingue les différentes P-métaheuristiques ?

- **génération** : comment générer la nouvelle population P' ?
- **sélection** : comment mettre à jour P (à partir de P et P') ?
- **mémoire** : quelle est l'information supplémentaire stockée et utilisée ?

Quelles sont les questions communes aux P-métaheuristiques ?

- comment déterminer la **population initiale** ?
- quels sont les **critères d'arrêt** ?

Population initiale

Comment déterminer la population initiale ?

Le choix de la population initiale peut jouer un rôle important pour l'efficacité en temps et pour la qualité des solutions trouvées.

Population initiale

Comment déterminer la population initiale ?

Le choix de la population initiale peut jouer un rôle important pour l'efficacité en temps et pour la qualité des solutions trouvées.

Principal critère de qualité : une population initiale devrait être **diversifiée** (sinon, risque de convergence prématurée).

Exemple : si on utilise k fois une heuristique gloutonne ou une S-métaheuristique pour générer chaque individu de la population initiale $\implies$ population très peu diversifiée.

Population initiale

Stratégies classiques pour la population initiale :

- **génération aléatoire** : en général, avec une probabilité uniforme en utilisant la génération de nombres pseudo-aléatoires
- **diversification séquentielle** : solutions générées en séquence de telle sorte que la diversification est favorisée (par ex., en imposant une distance¹ minimum entre un nouvel individu et les individus déjà générés).
- **diversification parallèle** : solutions générées d'une manière indépendante (en parallèle) (par ex. en décomposant l'espace d'états en zones distinctes et en générant aléatoirement une solution dans chaque zone).

1. Au sens des voisinages, voir définition d'un voisinage ci-avant.

Population initiale

Stratégies classiques pour la population initiale :

- **génération aléatoire** : en général, avec une probabilité uniforme en utilisant la génération de nombres pseudo-aléatoires
- **diversification séquentielle** : solutions générées en séquence de telle sorte que la diversification est favorisée (par ex., en imposant une distance¹ minimum entre un nouvel individu et les individus déjà générés).
- **diversification parallèle** : solutions générées d'une manière indépendante (en parallèle) (par ex. en décomposant l'espace d'états en zones distinctes et en générant aléatoirement une solution dans chaque zone).

1. Au sens des voisinages, voir définition d'un voisinage ci-avant.

Population initiale

Stratégies classiques pour la population initiale :

- **génération aléatoire** : en général, avec une probabilité uniforme en utilisant la génération de nombres pseudo-aléatoires
- **diversification séquentielle** : solutions générées en séquence de telle sorte que la diversification est favorisée (par ex., en imposant une distance¹ minimum entre un nouvel individu et les individus déjà générés).
- **diversification parallèle** : solutions générées d'une manière indépendante (en parallèle) (par ex. en décomposant l'espace d'états en zones distinctes et en générant aléatoirement une solution dans chaque zone).

1. Au sens des voisinages, voir définition d'un voisinage ci-avant.

Population initiale

Comparaisons des différentes stratégies

Stratégie	Diversité	Temps calcul	Qualité sol. initiales
Aléatoire	++	+++	+
Div. séquentielle	++++	++	+
Div. parallèle	++++	+++	+
Heuristique	+	+	+++

Pour chaque colonne, l'évaluation est meilleure s'il y a plus de +.

Critères d'arrêt

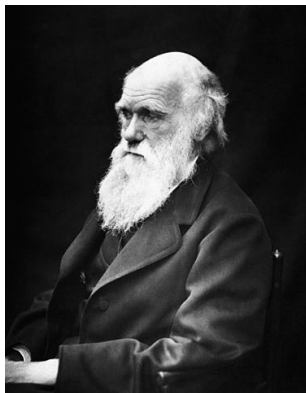
Quelles sont les conditions d'arrêt ?

- **condition statique** : la fin de la recherche est connue **a priori**. Par ex., un nombre fixé d'itérations (générations), une limite sur le temps CPU, ou un nombre maximum d'évaluations de la fonction objectif ;
- **condition adaptative** : on ne sait pas estimer quand la recherche sera terminée. Par ex., un nombre fixé d'itérations **sans amélioration**, conditions d'optimalité ou de satisfiabilité atteintes ;
- conjonction de plusieurs conditions ;
- conditions **spécifiques** à une P-métaheuristique donnée (souvent basées sur des mesures statistiques sur la population estimant le niveau de stagnation de celle-ci).

Algorithmes génétiques

- 19^{ième} siècle : Mendel pose les bases de la génétique (les lois de Mendel définissent la manière dont les gènes se transmettent de génération en génération).
- 1859 : Darwin énonce sa théorie de l'évolution.
- Depuis les années 1970 : les informaticiens s'inspirent de ces théories pour définir des algorithmes évolutifs, dont les algorithmes génétiques.

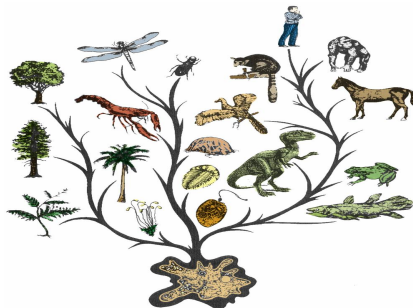
S'inspirer de la biologie ?



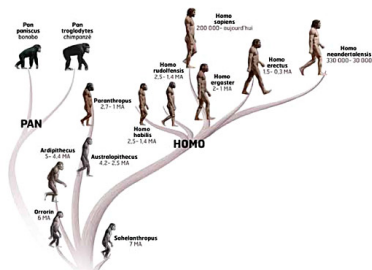
Charles Darwin (1809 – 1882)

Théories

- de l'évolution
- de la sélection naturelle



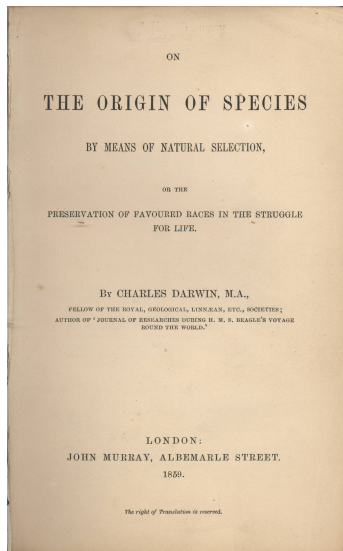
Théorie de l'évolution



« L'évolution est la transformation des espèces vivantes qui se manifeste par des changements de leurs caractères génétiques et morphologiques au cours des générations. Les changements successifs peuvent aboutir à la formation de nouvelles espèces. » (Wikipedia, 17 mars 2013)



Sélection naturelle



« L'évolution est causée, d'une part par la présence de *variations* parmi les traits héréditaires, tels que la couleur du plumage, d'une *population d'individus* lors des phases de *reproductions* avec l'apparition parfois de *mutations*, et d'autre part par divers mécanismes qui vont modifier la fréquence de certains traits héréditaires au sein de la population . . . »

(Wikipedia, 17 mars 2013)

Sélection naturelle



Fig. 34.—Skulls of Pigeons viewed laterally, of natural size. A. Wild Rock-pigeon, *Columba livia*. B. Short-faced Tumbler. C. English Carrier. D. Bagadocien Carrier.

« ... Parmi ces mécanismes, la **sélection naturelle** désigne la différence de propagation entre les traits héréditaires causée par leur effet sur la survie et la reproduction des individus. » (Wikipedia, 17 mars 2013)

Et la même chose dite plus simplement ?

Les mots-clefs de l'évolution

- variation d'une population d'individus via des
 - ▶ croisements (reproduction)
 - ▶ mutations

Et la même chose dite plus simplement ?

Les mots-clefs de l'évolution

- variation d'une population d'individus via des
 - ▶ croisements (reproduction)
 - ▶ mutations

Le mécanisme clef de la sélection naturelle

- les individus mieux adaptés (pour survivre à l'environnement) ont plus de chance de laisser une trace héréditaire au sein de la population

Et la même chose dite plus simplement ?

Les mots-clefs de l'évolution

- variation d'une population d'individus via des
 - ▶ croisements (reproduction)
 - ▶ mutations

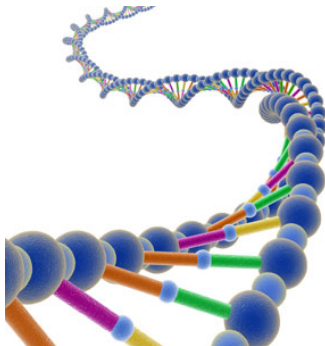
Le mécanisme clef de la sélection naturelle

- les individus mieux adaptés (pour survivre à l'environnement) ont plus de chance de laisser une trace héréditaire au sein de la population

L'ingrédient de base de la génétique

- le **code ADN** d'un individu

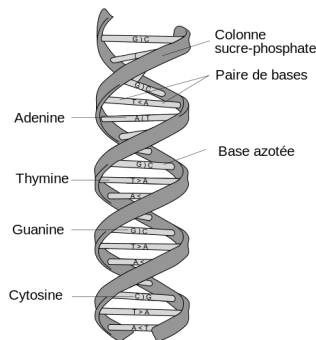
ADN



Acide désoxyribonucléique (ADN)

- molécule contenant les informations utiles pour faire fonctionner une cellule vivante
- support de l'hérédité (transmis partiellement lors de la reproduction)

ADN



[ACGTTACA ACTGGATCC]

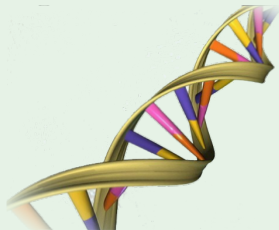
Structure de l'ADN

- double-hélice composée de deux « brins »
- chaque brin est une suite de nucléotides (**A**dénine, **G**uanine, **C**ytosine et **T**hymine)
- information (code) génétique = ordre des nucléotides ^a

a. On ne code qu'un seul brin car les nucléotides se regroupent toujours de la même façon : A avec T, T avec A, C avec G et G avec C.

Mais quel rapport avec nos problèmes, comme celui des n -reines ?

Biologie



Algorithme génétique

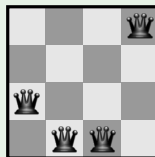


Un individu est une configuration

Biologie



Algorithme génétique

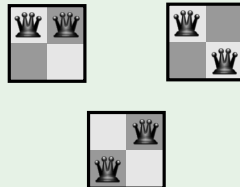


Une population est (un petit) groupe d'individus

Biologie



Algorithme génétique

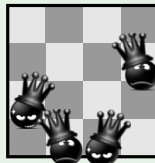
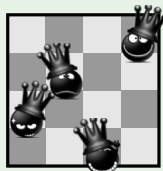


La santé d'un individu est (resp. inversement)
proportionnelle à la fonction objectif à maximiser (resp.
minimiser)

Biologie

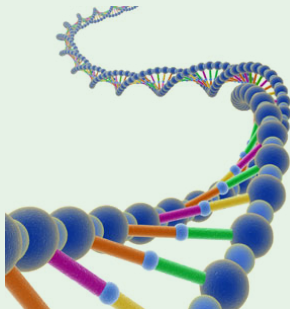


Algorithme génétique



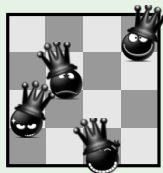
L'ADN d'un individu est sa représentation codée

Biologie



[ACGTTACG]

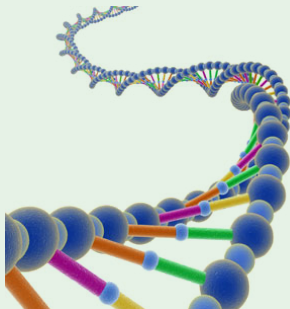
Algorithme génétique



Code ?

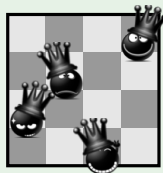
L'ADN d'un individu est sa représentation codée

Biologie



[ACGTTACG]

Algorithme génétique



Code ? [2, 3, 1, 4]

Un individu peut muter aléatoirement

Une mutation est un petit changement du code génétique

Biologie



Algorithme génétique



[1, 2, 1, 3]

Un individu peut muter aléatoirement

Une mutation est un petit changement du code génétique

Biologie



Algorithme génétique

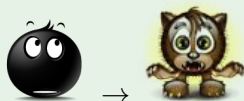


$[1, 2, 1, 3] \rightarrow [1, 4, 1, 3]$

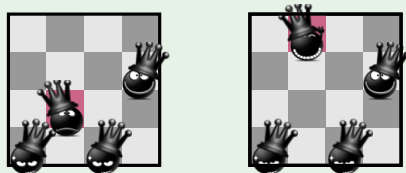
Un individu peut muter aléatoirement

Une mutation est un petit changement du code génétique

Biologie



Algorithme génétique



$[1, 2, 1, 3] \rightarrow [1, 4, 1, 3]$

Deux individus peuvent se reproduire

Les enfants reçoivent une partie du code génétique des deux parents (hérédité)

Biologie



Algorithme génétique



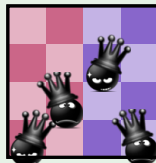
Deux individus peuvent se reproduire

Les enfants reçoivent une partie du code génétique des deux parents (hérédité)

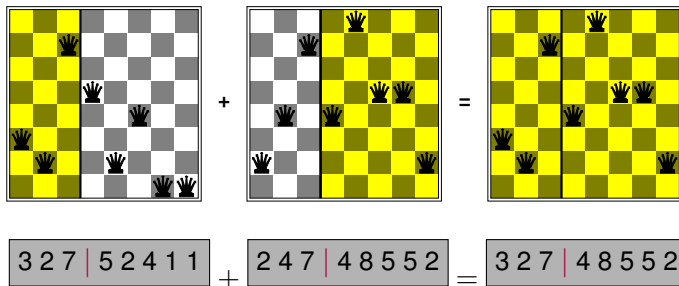
Biologie



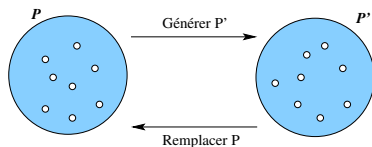
Algorithme génétique



Héritage du code génétique lors des croisements



Algorithme génétique complet



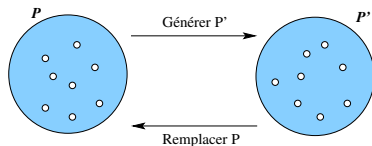
- générer une population initiale
- créer une nouvelle génération, tant qu'on a pas trouvé un individu « idéal »

- sélectionner des parents
- reproduction → enfants
- effectuer des mutations
- mettre à jour la population

Idée

Déterminer aléatoirement diverses configurations.

Algorithme génétique complet



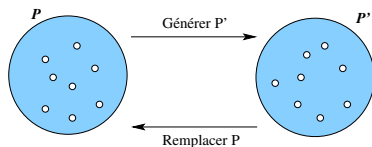
- générer une population initiale
- créer une nouvelle génération, tant qu'on a pas trouvé un individu « idéal »

- 1 sélectionner des parents
- 2 reproduction $\rightarrow$ enfants
- 3 effectuer des mutations
- 4 mettre à jour la population

Idée

A partir de P , générer P' . Ensuite, mettre à jour P à partir de P et P' .

Algorithme génétique complet



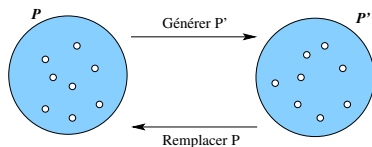
- générer une population initiale
- créer une nouvelle génération, tant qu'on a pas trouvé un individu « idéal »

- 1 sélectionner des **parents**
- 2 reproduction → **enfants**
- 3 effectuer des **mutations**
- 4 mettre à jour la population

Idée

Tout individu peut devenir parent, mais un parent en **bonne santé** a plus de chance d'être **sélectionné**.

Algorithme génétique complet

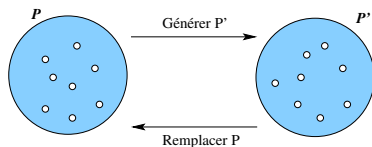


- générer une population initiale
- créer une nouvelle génération, tant qu'on a pas trouvé un individu « idéal »
 - 1 sélectionner des **parents**
 - 2 **reproduction** → **enfants**
 - 3 effectuer des **mutations**
 - 4 mettre à jour la population

Idée

Appliquer un **opérateur de croisement** sur le code génétique des parents. Les enfants se partagent le patrimoine génétique des parents (**hérédité**).

Algorithme génétique complet

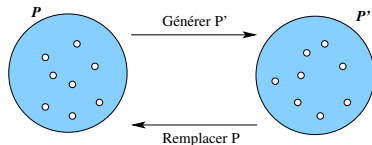


- générer une population initiale
- créer une nouvelle génération, tant qu'on a pas trouvé un individu « idéal »
 - 1 sélectionner des **parents**
 - 2 **reproduction** → **enfants**
 - 3 effectuer des **mutations**
 - 4 mettre à jour la population

Idée

Les mutations du code ADN des espèces s'observent dans la nature et sont à la base de la création de **nouvelles espèces**. Les mutations sont des **petits** changements et ne sont pas trop fréquentes.

Algorithme génétique complet



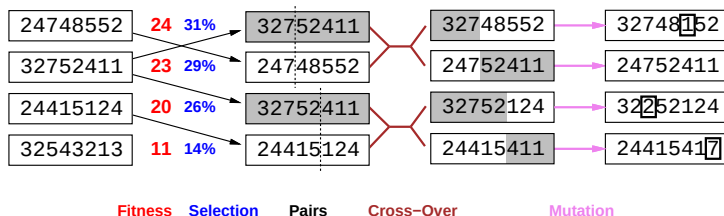
- générer une population initiale
- créer une nouvelle génération, tant qu'on a pas trouvé un individu « idéal »
 - 1 sélectionner des **parents**
 - 2 **reproduction** → **enfants**
 - 3 effectuer des **mutations**
 - 4 **mettre à jour** la population

Idée

Il faut décider qui va **survivre** pour garder une **taille constante** pour P . Un individu en bonne santé a plus de chance de survie (mais un individu plus faible peut survivre à un individu plus fort).

↪ algorithme stochastique : probabilités proportionnelles à la santé

Détail de deux générations successives



Concepts communs aux algorithmes génétiques

Outre les éléments communs à toutes les métaheuristiques (représentation, etc.) et P-métaheuristiques (population initiale, etc.), les algorithmes génétiques doivent définir

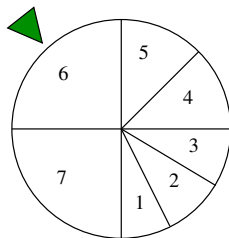
- une stratégie de **sélection**
- une stratégie de **reproduction**
- une stratégie de **remplacement**

Stratégies de sélection

- **La roulette** (roulette wheel selection) : assigner à chaque individu une probabilité de sélection qui est proportionnelle à sa santé relative (fitness, valeur de la fonction objectif). Soit f_i la santé d'un individu i , sa probabilité de sélection est

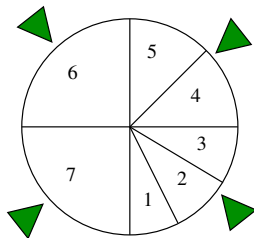
$$p_i = \frac{f_i}{\sum_{j=1}^n f_j}$$

Individus :	1	2	3	4	5	6	7
Fitness :	1	1	1	1.5	1.5	3	3



Stratégies de sélection

- Une variante de la roulette qui augmente la diversité : **Stochastic Universal Sampling**



Intuitivement : au lieu d'avoir 1 seul pointeur vers la "roulette", on en place plusieurs, uniformément distribués

Stratégies de reproduction : mutation

Les mutations sont des opérateurs de variation unaires.

- agissent sur un seul individu ;
- représentent un **petit** changement ;
- on définit une probabilité p_m de muter chaque élément (**gène**) d'une représentation (ou un seul gène, comme une seule valeur dans un vecteur) ;
- p_m est en général petite.

Utile : voisinages utilisés pour les S-métaheuristiques $\implies$ opérateurs de mutations

Stratégies de reproduction : mutation

Comme pour les voisinages, on privilégie un opérateur de mutation qui :

- permet d'atteindre n'importe quelle solution dans l'espace d'états ;
- produit des solutions **valides** (si possible) ;
- représente un changement **local**.

Stratégies de reproduction : croisement

Les croisements sont des opérateurs de variation binaires (et parfois n -aires).

- le rôle d'un croisement est d'hériter de certaines caractéristiques des 2 parents pour générer les descendants ;
- comme pour les mutations, la conception d'un opérateur de croisement dépend fortement des représentations utilisées ;
- un opérateur de croisement doit donner des solutions valides (si possible) ;
- on définit une probabilité qui représente la proportion des parents sur lesquels l'opérateur de croisement va agir.

Stratégies de reproduction : croisement

Exemple de croisement (représentation sous la forme de vecteurs) :

n-point crossover (= $n + 1$ segments aléatoires du vecteur recombinaés)

Parents								Descendants					
1	0	1	1	1	1		1	0	1	0	1	1	1
0	1	1	0	0	0		0	1	0	1	0	0	0

1-Point
crossover
 $\Rightarrow$

Stratégies de reproduction : croisement

Exemple de croisement (représentation sous la forme de vecteurs) :

n-point crossover (= $n + 1$ segments aléatoires du vecteur recombines)

Parents								Descendants					
1	0	1	1	1	1	0		1	0	1	0	0	0
0	1	1	0	0	0	1		0	1	1	1	0	1

2-Point
crossover
 $\Rightarrow$

Stratégies de reproduction : croisement

Exemple de croisement (représentation sous la forme de vecteurs) :

uniform crossover (chaque gène d'un descendant est sélectionné au hasard chez un de ses 2 parents. Chaque parent contribue équitablement à la génération des descendants)

Parents								Descendants						
1	1	1	1	1	1	1	uniform crossover ⇒	1	0	1	0	0	1	0
0	0	0	0	0	0	0		0	1	0	1	1	0	1

Stratégies de reproduction : croisement

Les exemples de croisements vus ci-avant sont applicables à des codages binaires ou des vecteurs de valeurs discrètes.

Pour des représentations codées sous la forme de **permutations**, ils ne sont plus applicables (les descendants $\neq$ permutations).

Parents							Descendants						
1	2	3	4	5	6	7	1	2	3	3	7	6	7
4	1	2	3	7	5	6	4	1	2	4	5	5	6

2-Point
crossover
 $\Rightarrow$

Stratégies de reproduction : croisement

Pour les **permutations**, il existe des croisements spécifiques.

Exemple : adaptation du 2-Point (les éléments de la section du milieu sont les mêmes que pour le parent *A* mais placés dans l'ordre du parent *B*. Les sections gauches et droites sont celles du parent *A*).

Parents							2-Point adapté ⇒	Descendants										
1	2		3	4	5			6	7	1	2		4	3	5		6	7
4	1		7	3	2			5	6	4	1		2	3	7		5	6

2-Point
adapté
⇒

Stratégies de remplacement

Taille n de la population P constante.

⇒ Comment sélectionner les survivants parmi l'ancienne population P et la nouvelle population P' ?

- **Remplacement générationnel** : P est remplacé par P' .
- **Remplacement stable** : à chaque génération, le meilleur descendant remplace le moins bon parent (variante : remplacer λ parents par λ descendants).
- **Elitisme** : sélectionner les n meilleurs individus parmi $P \cup P'$.
- Autres approches déterministes ou stochastiques.

Paramètres

Valeurs typiques (recommandées) pour les paramètres principaux des algorithmes évolutifs.

- **Probabilité de mutation** faible : $p_m \in [0.001, 0.01]$
- **Probabilité de croisement** (taux de croisement parmi P) moyenne ou élevée : $p_c \in [0.3, 0.9]$
- **Taille de la population** : entre 20 et 100 individus (une plus grande population augmente les chances de trouver de meilleures solutions mais complexité en temps croît linéairement avec la taille)

Autres types de P-métaheuristiques

Il existe quantité d'autres p-métaheuristiques² :

- autres types d'**algorithmes évolutifs** : Estimation of Distribution Algorithms, Differential Evolution, Coevolutionary Algorithms, Cultural Algorithms, Scatter Search, Path Relinking, etc.
- inspirées du **comportement de certaines espèces** : Swarm Intelligence algorithms (fourmis, abeilles, termites, poissons, oiseaux, etc.). Particularité : individus coopèrent (pour se nourrir) via un moyen indirect de communication (par ex. phéromones)
- mais encore : Artificial Immune Systems, etc.

2. Voir livre de référence *Metaheuristics : from design to implementation*.

Résumé du chapitre

Résumé du chapitre

Ce chapitre introduit la recherche locale et les métaheuristiques.

- un **algorithme de recherche locale** est utile quand seule la solution importe (et pas le chemin pour y arriver)
- les **problèmes d'optimisation** sont nombreux, souvent difficiles à résoudre, et seule la valeur de la solution optimale importe
- un algorithme de recherche locale simple, comme le **hill climbing** est sensible au problème des **optima locaux**
- les **métaheuristiques** sont des méthodes générales dont le but principal est de s'échapper d'optima locaux
- il existe 2 types de métaheuristiques : à **solution unique (S)** ou basée sur une **population (P)**

Résumé du chapitre

- toutes les métaheuristiques doivent définir : la **représentation** d'une solution ; la **fonction objectif** ; la gestion des **contraintes**
- les S-métaheuristiques impliquent de définir : un (ou plusieurs) **voisinage** ; une **solution initiale** ; une **évaluation incrémentale** du voisinage (si possible)
- les S-métaheuristiques se distinguent par les stratégies d'**intensification** et de **diversification**
- les P-métaheuristiques impliquent de définir : la **population initiale** et les **critères d'arrêt**
- les P-métaheuristiques se distinguent par les stratégies de **génération**, de **sélection** et de **mémoire**
- les métaheuristiques se basent souvent sur des principes **simples** ou **inspirés de la nature** et se révèlent efficaces pour de nombreux problèmes complexes